ТОО ”Колледж Хекслет”

Лабораторная работа №2

Маршрутизация и HTTP-методы в Express.js

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_2. Маршрутизация и HTTP-методы в Express.js.

Цель: Научиться создавать маршруты Express для разных HTTP-методов, понимать связь между HTTP-методом, URL и обработчиком, а также проверять работу REST API с помощью HTTP-клиента.

Условие: RO4_2/lesson2/ЛР-2.pdf

Требования задания

• запускаться командой npm start;

• использовать Express.js;

• содержать маршруты с методами GET, POST, PUT, PATCH и DELETE;

• иметь минимум три ресурса: products, categories, orders;

• использовать один и тот же URL /api/products с пятью различными HTTP-методами;

• возвращать JSON-ответы;

• использовать корректные HTTP-статусы для успешных запросов;

• содержать минимум один именованный обработчик маршрута;

• корректно работать при проверке через HTTP-клиент.

Ход выполнения

Для одного URL /api/products определены GET, POST, PUT, PATCH и DELETE. На этом этапе изменяющие методы возвращают сообщения-заглушки. Именованная функция getProducts передаётся Express как обработчик, а анонимная функция объявляется непосредственно при регистрации маршрута. Необъявленный DELETE /api/orders возвращает 404.

Проект: RO4_2/lesson2

Запуск из каталога проекта: npm install, затем npm start. Адрес сервера: http://localhost:3000.

app.js

app.js

```
import express from 'express';
import { pathToFileURL } from 'node:url';
const app = express();
let products = [
    {
        "id": 1,
        "name": "Keyboard",
        "price": 15000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 2,
        "name": "Mouse",
        "price": 8000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 3,
        "name": "Monitor",
        "price": 85000,
        "category": "Monitors",
        "inStock": false
    },
    {
        "id": 4,
        "name": "Laptop",
        "price": 420000,
        "category": "Computers",
        "inStock": true
    },
    {
        "id": 5,
        "name": "Webcam",
```

Листинг 1. app.js (часть 1)

app.js

```
        "price": 22000,
        "category": "Accessories",
        "inStock": false
    }
];
let nextId = 6;
const categories = ['Computers', 'Accessories', 'Monitors'];
app.get('/', (req, res) => res.send('Internet Shop API'));
app.get('/about', (req, res) => res.send('Учебный REST API магазина компьютерной
техники'));
app.get('/api/status', (req, res) => res.json({ status: 'ok', message: 'Server is
running' }));
app.get('/api/shop', (req, res) => res.json({ name: 'Kirill Tech Store', description:
'Учебный интернет-магазин', currency: 'KZT' }));
function getProducts(req, res) { res.json(products); }
app.get('/api/products', getProducts);
app.get('/api/categories', (req, res) => res.json(categories));
app.post('/api/products', (req, res) => res.status(201).json({ message: 'POST route:
product creation' }));
app.put('/api/products', (req, res) => res.status(200).json({ message: 'PUT route: full
product replacement' }));
app.patch('/api/products', (req, res) => res.status(200).json({ message: 'PATCH route:
partial product update' }));
app.delete('/api/products', (req, res) => res.status(200).json({ message: 'DELETE route:
 product deletion' }));
app.post('/api/categories', (req, res) => res.status(201).json({ message: 'POST route:
category creation' }));
app.get('/api/orders', (req, res) => res.json([]));
app.post('/api/orders', (req, res) => res.status(201).json({ message: 'POST route: order
 creation' }));
app.get('/api', (req, res) => res.json({ name: 'Internet Shop API', version: '1.0',
resources: ['/api/products', '/api/categories', '/api/orders'] }));
export default app;
if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
    const PORT = Number(process.env.PORT || 3000);
```

Листинг 2. app.js (часть 2)

app.js

```
    app.listen(PORT, () => console.log(`Server: http://localhost:${PORT}`));
}
```

Листинг 3. app.js (часть 3)

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Обязательные endpoints

• HTTP-статусы и содержимое ответов

HTTP-запросы

GET / → 200

GET /about → 200

GET /api/status → 200

GET /api/shop → 200

GET /api/products → 200

GET /api/categories → 200

GET /api → 200

GET /api/orders → 200

POST /api/products → 201

PUT /api/products → 200

PATCH /api/products → 200

DELETE /api/products → 200

POST /api/categories → 201

POST /api/orders → 201

DELETE /api/orders → 404

Примеры ответов

POST /api/products → 201


```
{
  "message": "POST route: product creation"
}
```


PATCH /api/products → 200


```
{
  "message": "PATCH route: partial product update"
}
```


DELETE /api/products → 200


```
{
  "message": "DELETE route: product deletion"
```

}

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.